

基於自動爬蟲與大型語言模型的 AI 點餐推薦系統

專題研究期末計畫書

指導教師：楊朝棟

組員：盧袁聖 / 林亮宇

系級：資工三 A

中華民國 115 年

摘要

本研究設計並實作一套「AI 點餐推薦系統」，目標是解決使用者在日常用餐情境中，必須同時考量預算、辣度、忌口、飲料需求、用餐人數與菜品類型時，難以快速從餐廳菜單中做出選擇的問題。系統以自動化網路爬蟲取得餐廳菜單資料，再透過大型語言模型與規則式偏好抽取方法理解使用者以自然語言輸入的需求，最後由推薦演算法選出合適的餐點組合，並以自然語言產生點餐建議。

目前專案採用 Python 作為主要開發語言，後端以 FastAPI 建立 API 服務，前端提供類 ChatGPT 的對話介面，並支援對話紀錄、餐廳切換、完整菜單檢視與新餐廳菜單爬取。菜單取得部分使用 Playwright 連接 Chrome 遠端調試模式，從 Google 搜尋結果中尋找餐廳菜單按鈕並擷取品項名稱與價格；若文字菜單無法取得，系統改以 Google Images 搜尋菜單圖片，再交由視覺語言模型判斷圖片內容，解析其中的菜單品項與價格，並進行二次校驗。語言模型推論部分以 Ollama 執行本地端模型 Gemma3-12B 為主，也保留 OpenAI-compatible API 介面以支援外部文字或視覺模型。

本研究的核心貢獻在於整合「即時菜單取得」、「多輪對話偏好記憶」與「預算約束下的點餐組合推薦」。預期成果包含：提升使用者選餐效率、降低人工建置菜單資料庫的成本、建立可擴充的多餐廳點餐推薦架構，並驗證本地端大型語言模型在日常推薦任務中的可行性。

關鍵字：點餐推薦、對話式推薦系統、大型語言模型、Playwright、FastAPI、Ollama、視覺語言模型

1 研究動機與研究問題

1.1 研究動機

在日常生活中，「今天要吃什麼」看似簡單，實際上常涉及多個限制條件。使用者可能同時具有預算上限、口味偏好、忌口食材、是否需要飲料、同行人數、是否想吃主餐或甜點等需求。傳統菜單多以靜態圖片或文字呈現，使用者需要自行閱讀、比價、排除不適合品項，當菜單項目數量較多時，決策成本會明顯上升。

現有外送平台或餐廳 App 多半仰賴平台內部結構化資料庫，推薦方式常以熱門、距離或平台促銷為主，較少能針對「自然語言輸入的複合條件」做即時推理。例如使用者輸入「兩個人吃，預算 500，不要辣，想要一份主餐加飲料」，系統需要同時理解人數、預算、辣度、餐點類型與飲料需求，而非只做關鍵字搜尋。

近年大型語言模型在語意理解與自然語言生成上快速發展。Gemma 3 技術報告指出，Gemma 3 屬於多模態開放模型系列，支援文字與視覺理解，並以較輕量的方式支援長上下文與多語能力 [1]。這使得本地部署模型處理日常中文任務變得可行。本研究因此嘗試將自動化爬蟲、視覺語言模型與對話式推薦整合，建立一套能動態取得菜單並根據使用者需求推薦餐點的 AI 點餐助理。

1.2 研究問題

本研究聚焦於以下研究問題：

1. 如何透過自動化爬蟲從 Google 搜尋結果中擷取餐廳菜單，並在 Google 頁面結構變動、反爬蟲驗證、菜單圖片化等情況下維持可用性？
2. 如何將使用者的自然語言需求轉換成可運算的偏好狀態，例如預算、辣度、忌口、菜品類型、飲料需求與用餐人數？
3. 如何在多輪對話中保留先前偏好，使後續輸入能更新而非覆蓋全部狀態？
4. 如何在預算限制下從菜單中組合主餐、配菜、飲料與甜點，並產生自然且可理解的推薦理由？
5. 若使用本地端 LLM 取代雲端 API，系統在隱私保護、部署成本與回覆品質之間能否取得合理平衡？

2 文獻回顧與探討

2.1 大型語言模型與本地端推論

大型語言模型能透過上下文理解使用者意圖，適合處理非結構化、口語化的點餐需求。Gemma 3 作為 Google 發表的開放模型系列，強調多模態、長上下文與多語能力 [1]，適合用於中文對話理解、偏好抽取與自然語言回覆生成。本研究採用 Gemma3-12B 作為主要文字模型，透過 Ollama 進行本地端推論。Ollama 的優點是部署流程簡單，可直接以命令列或本地服務執行模型，降低將使用者對話傳送至外部雲端服務的需求 [2]。

本地端模型也有其限制。第一，模型推論速度受限於使用者電腦硬體；第二，模型可能對菜名分類、價格估算或否定詞理解產生錯誤。因此本系統未完全依賴 LLM，而是採用「規則式抽取 + LLM 輔助」的混合設計。規則負責穩定處理預算、辣度、否定詞與常見菜品類型，LLM 則負責語意較複雜的分類與回覆生成。

2.2 對話式推薦系統

透過多輪互動逐步補齊使用者偏好。相關研究指出，CRS 的核心在於透過對話取得使用者需求，並在互動過程中持續更新偏好狀態，使推薦結果能隨著使用者補充條件而調整 [7, 8]。此外，近期整體式對話推薦研究也強調，真實對話中的使用者意圖可能不完整或持續變化，因此系統需要能在多輪互動中記錄、修正與合併使用者偏好 [9, 10]。

本研究的點餐情境與 CRS 高度相關，因為使用者通常不會一次完整輸入所有條件，而是會逐步補充，例如先說「想吃漢堡」，接著補充「不要飲料」、「預算 200 以內」。因此，本系統在後端保留每個對話 session 的偏好狀態，讓新輸入可以更新既有條件，而不是每一輪都重新開始推薦。

2.3 自動化爬蟲與視覺語言模型

餐廳菜單常出現在 Google 搜尋結果、Google Maps 資訊欄或圖片搜尋結果中，頁面通常由 JavaScript 動態渲染。Playwright 是 Microsoft 維護的瀏覽器自動化框架，適合

操作動態網頁、等待元素載入與擷取頁面內容 [6]。本研究利用 Playwright 連接 Chrome 的 CDP 遠端調試模式，使爬蟲能操作真實瀏覽器環境，並保留使用者登入或驗證狀態。

此外，許多餐廳未提供結構化文字菜單，只會在 Google 評論、社群平台或店面照片中出現菜單圖片。菜單圖片辨識不只是單純讀取文字，還需要判斷哪些圖片是真正的菜單，哪些只是食物照片、店面照片或外送平台商品卡。CLIP 類模型證明了影像與文字可透過共同表示空間進行連結，而 LLaVA 等視覺語言模型則進一步展示了模型可根據圖片與文字指令進行多模態理解 [11, 12]。因此，本研究使用視覺語言模型判斷菜單圖片內容，並解析其中的品項名稱與價格。

2.4 相關技術比較

表 1 相關技術與本研究比較

方法或系統	資料來源	推薦方式	與本研究差異
傳統外送平台	平台內部資料庫	熱門、距離、評分	缺少自然語言複合條件理解，也較依賴平台既有資料
一般 OCR 菜單辨識	菜單圖片	文字擷取	可取得文字，但不負責偏好理解與點餐組合推薦
對話式推薦系統	使用者對話	多輪偏好建模	多數未結合即時菜單爬取與圖片菜單備援
本研究	Google 文字菜單與圖片菜單	LLM + 規則式抽取 + 預算控制	同時整合菜單取得、偏好記憶、推薦組合與自然語言回覆

3 研究方法及步驟

3.1 系統架構

本系統可分為四個層級：使用者介面層、應用服務層、AI 推薦層與資料取得層。分層式架構如圖 1 所示。此架構將使用者互動、後端 API、推薦邏輯與爬蟲資料來源分離，使系統後續能獨立替換模型、資料庫或前端介面。



圖 1 AI 點餐推薦系統分層架構圖

系統實際資料流則如圖 2 所示。使用者在前端輸入需求後，前端呼叫 `/api/chat`，後端根據 session 取得累積偏好，推薦引擎再讀取目前活動餐廳菜單並產生回覆。若使用者新增餐廳，後端會觸發 Playwright 爬蟲，將取得的菜單轉為 JSON 後加入多餐廳管理模組。

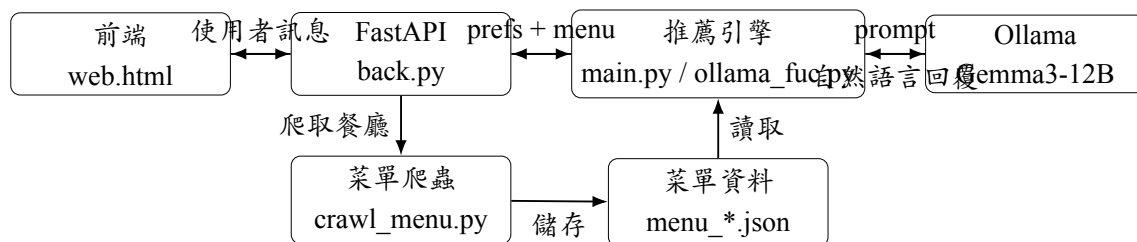


圖 2 系統資料流圖

3.2 系統操作流程

系統操作流程如圖 3 所示。使用者可先選擇既有餐廳，或輸入新餐廳名稱觸發爬蟲。取得菜單後，使用者以自然語言輸入需求，系統會抽取偏好、更新 session 狀態、進行餐點推薦，最後輸出自然語言點餐建議。

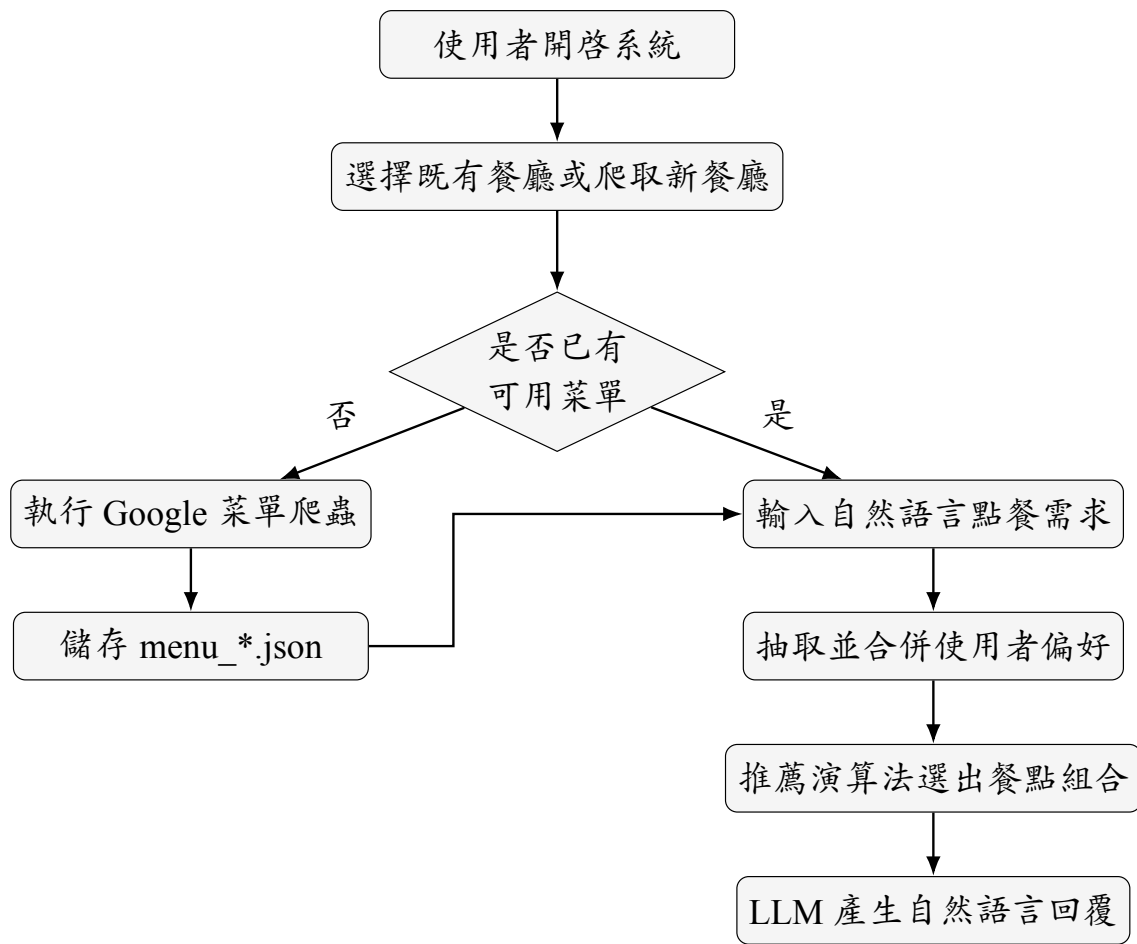


圖 3 系統操作流程圖

3.3 菜單爬蟲設計

菜單爬蟲位於 `crawl_menu.py`，主流程可分為文字菜單擷取與圖片備援兩條路徑，如圖 4 所示。

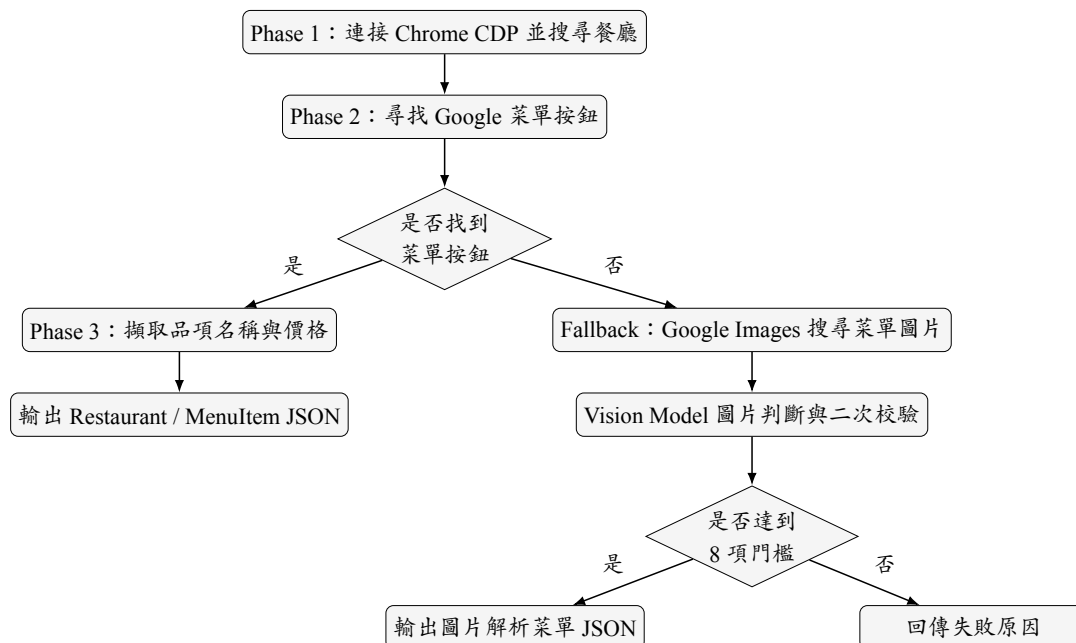


圖 4 菜單爬蟲與圖片備援流程圖

文字菜單部分，系統先檢查本機 9222 連接埠是否已有 Chrome 遠端調試服務。若沒有，爬蟲會嘗試啟動 Chrome，並指定 `--remote-debugging-port=9222` 與專案下的 `.chrome_debug_profile` 作為持久化使用者資料夾。連線成功後，Playwright 導向 Google 搜尋結果頁，依序嘗試右側資訊欄、全頁按鈕與導覽列文字來尋找「菜單」按鈕。

圖片備援部分，若文字菜單無法取得，系統改用 Google Images 搜尋餐廳圖片，根據關鍵字、圖片尺寸、長寬比、版面位置與解析度對候選圖片評分。排序後的候選圖片交由視覺模型解析，模型必須回傳嚴格 JSON 格式，且至少解析出 8 個品項才視為有效。

3.4 偏好理解模組

偏好理解模組的目標是將自然語言轉為可被推薦演算法使用的狀態。系統支援的偏好欄位如表 2 所示。

表 2 偏好欄位與抽取方式

偏好欄位	範例	抽取方式
預算	200 元、不超過 500	正規表示式擷取數字
辣度	不辣、微辣、小辣	關鍵字比對
忌口	不吃牛肉、不要花生	觸發詞後方片段切分
菜系	日式、美式、義式	關鍵字比對
菜品類型	漢堡、吐司、貝果、套餐	關鍵字與英文別名比對
飲料需求	要飲料、不要飲料	先判斷否定詞，再判斷肯定詞
用餐人數	2 人、4 人	正規表示式擷取整數

偏好抽取採雙層設計。第一層是規則式抽取，處理格式明確且需穩定判斷的條件，例如預算與否定詞。第二層是可選的 LLM 抽取，當環境變數 `USE_LLM_EXTRACTION=true` 時啟用，模型會被要求只輸出 JSON。若 LLM 回覆無法解析，系統會降級回規則式結果。

多輪對話部分，系統將每個 session 的 `prefs` 與 `history` 存在後端記憶體中。使用者每次輸入後，系統先抽取出本輪偏好，再透過 `merge_prefs_inplace()` 合併到原本偏好狀態。這代表新輸入可以更新預算、辣度或飲料需求，但未重新提到的舊偏好仍會保留。

3.5 推薦演算法

推薦引擎主要在 `src/ollama_fuc.py` 的 `recommend()` 中實作。流程如下：

1. 菜單展平：支援多餐廳格式 `restaurants` 與舊版 `categories` 格式，將所有品項展平成 `name`、`price`、`category`、`restaurant`。
2. 忌口過濾：若使用者輸入「不要牛肉」或辣度為「不辣」，系統會把相關關鍵字加入排除列表。
3. 價格正規化：若價格為字串，系統以正規表示式取出數字；若缺失或無法解析，則以極大值排序。
4. 菜品分類：系統可使用 LLM 批次分類菜品為 `main`、`side`、`drink`、`dessert`、`other`。若 LLM 失敗，則使用關鍵字分類。
5. 偏好匹配：若使用者指定漢堡、吐司、貝果或套餐，系統會優先選取符合該類型的主餐。
6. 預算控制：系統優先選 1-2 個主食，第一個主食原則上不超過預算的 40%，主食總額不超過約 65%；配菜最多使用到預算約 90%；飲料與甜點則需避免總額超過預算。
7. 回覆生成：推薦品項與預算估算會組成 JSON，再交給 Gemma3-12B 產生自然中文回覆。若 LLM 失敗，系統使用備用模板回覆，避免服務中斷。

3.6 資料格式

爬蟲取得菜單後，會將資料整理為 JSON 檔案。簡化後的資料格式如下：

```
{
  "name": "restaurant_name",
  "menu_items": [
    {"name": "burger", "price": "120"},
    {"name": "black_tea", "price": "30"}
  ]
}
```

後端會再將此格式轉換為多餐廳管理格式，使前端能切換不同餐廳，推薦引擎也能知道目前使用者正在查詢哪一間餐廳。

4 系統實作與功能

4.1 後端 API

後端採用 FastAPI。FastAPI 支援 Python 型別註記與 Pydantic schema，可快速建立 API，並自動產生 OpenAPI 文件 [3, 4, 5]。本系統目前主要 API 如表 3 所示。

表 3 主要 API 功能

API	方法	功能
/health	GET	確認後端服務是否運行
/	GET	回傳前端 web.html
/api/chat	POST	接收使用者訊息並回傳推薦回覆
/api/current-menu	GET	回傳目前活動餐廳的完整菜單
/api/restaurants	GET	列出已載入餐廳與品項數
/api/switch-restaurant	POST	切換目前活動餐廳
/api/menu/{restaurant_name}	DELETE	刪除指定餐廳菜單
/api/update-menu	POST	遠端觸發爬蟲更新餐廳菜單

後端除了處理一般 API 請求，也負責遠端觸發菜單爬蟲。當使用者在前端輸入新餐廳名稱並呼叫 /api/update-menu 時，後端會啟動 Playwright 爬蟲流程。Playwright 是 Microsoft 維護的瀏覽器自動化框架，支援 Chromium、Firefox 與 WebKit，適合操作動態網頁、等待元素載入與擷取頁面內容 [6]。本系統利用 Playwright 連接 Chrome 遠端調試模式，從 Google 搜尋結果中尋找餐廳菜單按鈕；若無法取得文字菜單，則改以 Google Images 搜尋菜單圖片，並交由視覺語言模型判斷圖片內容與解析菜單品項。

此外，後端也處理 Windows 平台上的 `asyncio event loop` 相容問題。由於 Playwright 爬蟲在 Windows 上需要 `ProactorEventLoop`，而 Uvicorn 服務本身可能使用不同事件迴圈，因此系統將爬蟲放入獨立 `thread`，並在該 `thread` 中建立新的 `event loop` 執行。此設計可降低後端 API 與爬蟲非同步流程互相衝突的風險。

4.2 使用者介面展示

前端採單頁 HTML、CSS 與 JavaScript 實作，不需要額外前端框架。介面包含左側對話列表、餐廳選擇器、匯入匯出、釘選、重新命名與刪除對話功能；主要區域則是對話框與輸入框。使用者送出訊息後，前端會把 `sessionId` 與文字傳送至後端。每個對話 `thread` 擁有獨立 `sessionId`，因此不同對話可以維持不同偏好狀態。



圖 5 AI 點餐推薦系統主對話介面



圖 6 完整菜單檢視介面

此外，前端支援查看完整菜單與爬取新餐廳菜單。當使用者點選「爬菜單」後，可輸入餐廳名稱並呼叫後端爬蟲 API；爬蟲成功後，新的餐廳菜單會加入餐廳列表並設為活動餐廳，使用者即可直接開始詢問推薦。

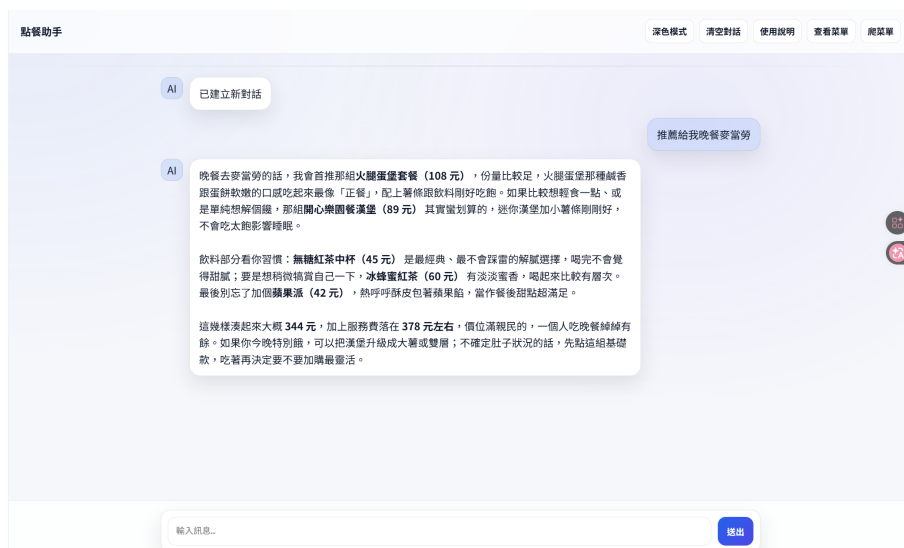


圖 7 AI 點餐推薦結果畫面

圖 7 為推薦結果畫面。系統會根據使用者輸入的預算、忌口、辣度與餐點類型，從目前餐廳菜單中挑選合適品項，並以自然語言說明推薦理由。

4.3 資料儲存與多餐廳管理

系統支援預設菜單 menu.json 與爬蟲產生的 menu_餐廳名.json。啟動時後端會掃描專案根目錄下所有 menu_*.json，將其轉換為統一的多餐廳格式後存入 RESTAURANT_MENUS。目前活動餐廳由 ACTIVE_RESTAURANT 控制，前端可以透過 API 查詢、切換或刪除。

此設計的好處是不用每次重啟服務才能切換餐廳，也不需要一開始就建立大型資料庫。對專題研究而言，JSON 檔案足以支援功能驗證；未來若要擴充成正式服務，可再改接 SQLite、MySQL 或其他資料庫。

5 系統測試與預期結果

5.1 系統測試案例

為了驗證系統是否能處理自然語言需求，本研究設計數組測試輸入，觀察系統是否能正確抽取偏好並產生推薦。測試案例如表 4 所示。

表 4 系統測試案例

使用者輸入	系統解析偏好	預期推薦行為
我想吃漢堡，200 元以內，不要飲料	菜品類型：漢堡；預算：200；飲料：否	優先推薦漢堡類主餐，不加入飲料，並控制在預算內
兩個人吃，預算 500，不辣	人數：2；預算：500；辣度：不辣	排除含辣品項，推薦主餐與配菜組合
我不吃牛肉，想要有飲料	忌口：牛肉；飲料：是	排除牛肉品項，並在預算允許時加入飲料
想吃套餐，300 元以下	菜品類型：套餐；預算：300	優先選擇套餐或主食類品項，避免超出預算

5.2 功能性目標

本研究預期完成後，系統應具備以下功能性成果：

1. 使用者可透過網頁對話介面輸入自然語言需求，並收到完整點餐建議。
2. 系統可從 Google 搜尋結果自動取得餐廳文字菜單；若文字菜單無法取得，能改用圖片搜尋與視覺模型作為備援。
3. 系統可解析預算、辣度、忌口、菜品類型、飲料需求與用餐人數等偏好。
4. 系統可在多輪對話中保留偏好狀態，例如先指定菜品類型，再補充預算與忌口。
5. 推薦結果需包含主餐優先、預算控制與推薦理由，而不是只列出菜單項目。
6. 使用者可新增、切換、刪除餐廳菜單，並查看目前活動餐廳的完整菜單。

5.3 評估指標

由於不同餐廳在 Google 上的資料完整度差異很大，本研究不宜只用單一餐廳測試。後續可建立 20 至 30 間餐廳的測試集，依表 5 的指標評估。

表 5 預期評估指標

指標	衡量方式	目標
菜單取得成功率	成功取得 8 項以上菜單品 項的餐廳比例	60% 以上
偏好抽取正確率	人工標註使用者輸入，再 比對系統抽取結果	80% 以上
多輪偏好保留率	第三輪後檢查第一輪偏好 是否仍存在	90% 以上
預算符合率	推薦總額是否不超過使用 者預算	95% 以上
平均回應時間	從送出訊息到收到回覆	30 秒內
備援成功率	文字菜單失敗後，圖片備 援成功取得菜單比例	可量化紀錄

5.4 非功能性目標與限制

非功能性方面，本系統重視隱私、可維護性與容錯性。使用 Ollama 本地端模型時，使用者文字對話不需傳送至雲端模型服務；爬蟲與推薦模組分離，也讓未來更換資料來源或模型時不必重寫整個系統。系統同時保留模板式 fallback，當 LLM 逾時、模型不存在或回覆空白時，仍可產生基本推薦文字。

限制方面，Google 搜尋結果並非穩定 API，CSS 選擇器可能變動，也可能觸發人機驗證；圖片菜單的辨識品質受圖片解析度、拍攝角度與菜單版面影響；LLM 對菜品分類與自然語言回覆可能出現不一致。因此正式部署前，仍需要加入更完整的錯誤紀錄、人工校正流程與資料來源合法性檢查。

6 研究流程規劃

本研究以一學期約 18 週規劃，主要工作包含文獻整理、環境建置、爬蟲開發、偏好理解、推薦演算法、前後端整合、視覺備援、測試評估與報告撰寫。研究時程如圖 8 所示。

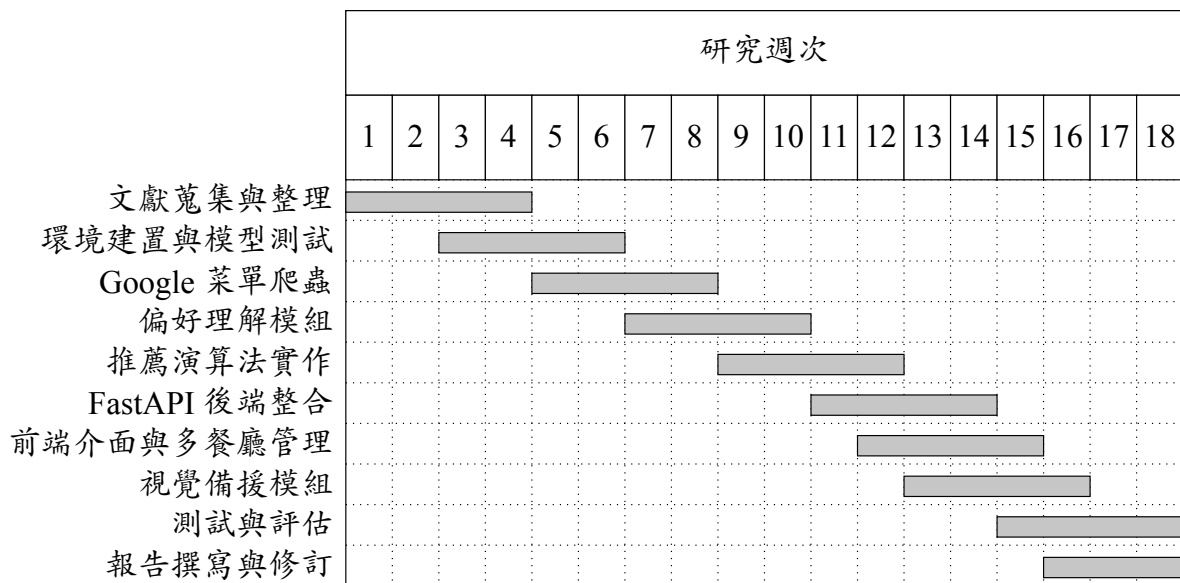


圖 8 研究時程甘特圖

主要里程碑如下：第 6 週完成基本爬蟲流程，能取得至少數間餐廳菜單；第 10 週完成偏好抽取與初版推薦演算法；第 14 週完成前後端整合與多餐廳管理；第 16 週完成圖片備援與視覺模型測試；第 18 週完成系統展示、測試紀錄與期末報告。

7 結論

本研究提出一套結合自動化爬蟲、本地端大型語言模型、視覺備援與對話式推薦的 AI 點餐推薦系統。相較於只依賴人工資料庫或靜態菜單搜尋的方式，本系統可從 Google 搜尋結果動態取得餐廳菜單，並透過多輪對話逐步累積使用者偏好。推薦演算法則將菜單品項分為主餐、配菜、飲料與甜點，並在預算限制下組合出較符合真實點餐情境的建議。

目前系統已具備完整雛形，但仍需擴充測試樣本、量化爬蟲成功率、改善視覺語言模型對菜單圖片的判斷穩定度，並處理 Google 反爬蟲與資料來源合規問題。後續若能加入人工校正菜單、資料庫儲存、使用者評分回饋與更完整的實驗設計，系統將更接近可實際部署的個人化點餐助理。

參考文獻

- [1] Gemma Team. *Gemma 3 Technical Report*. arXiv:2503.19786, 2025.
- [2] Ollama Team. *Ollama Documentation*. Available: <https://ollama.com/>
- [3] Ramírez, S. *FastAPI Documentation*. Available: <https://fastapi.tiangolo.com/>
- [4] Pydantic Team. *Pydantic Documentation*. Available: <https://docs.pydantic.dev/>
- [5] OpenAPI Initiative. *OpenAPI Specification*. Available: <https://spec.openapis.org/oas/latest.html>
- [6] Microsoft. *Playwright Documentation*. Available: <https://playwright.dev/>
- [7] Jannach, D., Manzoor, A., Cai, W., and Chen, L. “A Survey on Conversational Recommender Systems.” arXiv:2004.00646, 2020.
- [8] Gao, C., Lei, W., He, X., de Rijke, M., and Chua, T.-S. “Advances and Challenges in Conversational Recommender Systems: A Survey.” arXiv:2101.09459, 2021.
- [9] Li, C., Hu, H., Zhang, Y., Kan, M.-Y., and Li, H. “A Conversation is Worth A Thousand Recommendations: A Survey of Holistic Conversational Recommender Systems.” arXiv:2309.07682, 2023.
- [10] Jannach, D. “Evaluating Conversational Recommender Systems: A Landscape of Research.” arXiv:2208.12061, 2022.
- [11] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. “Learning Transferable Visual Models From Natural Language Supervision.” *Proceedings of the 38th International Conference on Machine Learning*, 2021.
- [12] Liu, H., Li, C., Wu, Q., and Lee, Y. J. “Visual Instruction Tuning.” arXiv:2304.08485, 2023.